

FAST-MVH: Geometric Indexing for Dominance Checking with Multi-Valued Heuristics

Abstract

Multi-valued heuristics (MVHs) supply several lower-bound vectors per state, each describing a possible trade-off among the objectives. Combined with dimensionality reduction, they reduce the number of paths a multi-objective search must expand. The price is dominance checking. A generated path is tested against the frontier of its state, each candidate heuristic vector is tested against the solutions found so far, and an exact fallback must sometimes consult vectors that have left the frontier. We present FAST-MVH, which answers all of these queries with geometric indexes. Truncated frontiers are stored in adaptive k-d trees. Local dominance is decided in a single traversal and is tested before a heuristic is selected. The exact fallback is confined to a residue set that we prove sufficient and index with a forest of static k-d trees. FAST-MVH expands the same paths as $L\text{-NAMOA}_{dr}^*\text{-mvh}$ and returns the same solutions. On seventeen grid and road-network instances with three to eight objectives it runs 1.8 to 82 times faster, with byte-identical solution sets in every case.

1 Introduction

A shortest path need not be best in every objective. One route is shorter, another faster, a third cheaper. Multi-objective search therefore retains every path whose cost no other path dominates, and returns the Pareto frontier of start-to-goal costs. The retained sets grow quickly with the number of objectives M . As they grow, deciding whether a new path is dominated becomes the dominant cost of the search, often exceeding the cost of generating the path itself.

A second difficulty is weak guidance. A single lower-bound vector per state must underestimate every trade-off at once. *Multi-valued heuristics* (MVHs) instead give a set of vectors, each a lower bound on some of the remaining trade-offs (Geisser et al. 2022; Skyler et al. 2024). The information is richer, but using it is not free: a path may test several heuristic vectors before it finds one that the solutions found so far do not refute.

Wolff, Felner, and Salzman (2026) combine MVHs with dimensionality reduction in $L\text{-NAMOA}_{dr}^*\text{-mvh}$. Each path carries one active heuristic vector. When a new solution refutes it, the search selects the next vector and reinserts the path. Dominance is tested on the last $M - 1$ coordinates whenever that suffices and on all M coordinates otherwise. Three queries recur: *goal dominance* of a path’s f -value, *local dominance* of its g -value at its state, and *heuristic selection*, which is a sequence of goal-dominance queries.

Each query is an orthant-emptiness test: does a set contain a vector in the lower orthant of the query point? Shperberg et al. show that structures keyed by a single lexicographic order need n dominance tests on an n -vector antichain when $M \geq 4$, whereas a geometric index answers the query in $O(n^{1-1/d})$ for

$d = M - 1$. FAST-MVH builds on this observation. Its contributions are:

1. **Adaptive truncated frontiers.** Each state frontier starts as a contiguous array and is promoted to a dynamic k-d tree over its last $M - 1$ coordinates only when its scans become expensive.
2. **A single-pass local test, applied first.** One traversal of the frontier classifies a path as undominated, fully dominated, or truncation-dominated only. At generation, this test precedes heuristic selection.
3. **An indexed exact fallback.** The rare full-dimensional fallback searches a *residue* of evicted vectors, which we prove sufficient. The residue is append-only and is indexed by the logarithmic method.
4. **A negative result.** Indexing the heuristic sets themselves does not pay. We explain why.

2 Background

Let $G = (S, E, c)$ be a finite graph with $c(e) \in \mathbb{R}_{\geq 0}^M$. Path costs are component-wise sums. A vector x weakly dominates y , written $x \leq y$, if $x_k \leq y_k$ for every k . The task is to return one path for each distinct non-dominated start-to-goal cost. We write $\leq_{\text{lex}}$ for lexicographic order and

$$\text{Tr}(x) = (x_2, \dots, x_M).$$

Multi-valued heuristics. An admissible MVH assigns a finite set $H(s)$ to each state such that for every goal-reaching suffix π from s , some $h \in H(s)$ satisfies $h \leq c(\pi)$. We assume $H(s_{\text{goal}}) = \{0\}$ and, following Wolff et al., the order

$$H(s) = \langle h^1, \dots, h^{k(s)} \rangle, \quad h^1 \leq_{\text{lex}} \dots \leq_{\text{lex}} h^{k(s)}.$$

Under dimensionality reduction this order is a precondition of correctness, not a traversal preference: a path must use the first member of $H(s)$ that the goal frontier does not refute. No index may change that answer.

Example. Two suffixes from s cost $(1, 10)$ and $(10, 1)$. The set of these two vectors is an admissible MVH, but its component-wise maximum $(10, 10)$ bounds neither suffix. Landmark bounds (ALT) hold for *every* suffix, so their maximum is admissible and a set of them is better replaced by its maximum. Genuine MVHs, such as backward Pareto sets, lack this property; replacing them by their maximum destroys admissibility.

Search state. A node n records a state $s(n)$, cost $g(n)$, heuristic index $i(n)$, and $f(n) = g(n) + h^{i(n)}$. OPEN is ordered lexicographically by f . Let $C(s)$ be the costs of paths expanded at s . The *reduced frontier* $F(s) \subseteq C(s)$ stores full vectors but is compared on truncations. Expanding g at s performs

$$F(s) \leftarrow \{p \in F(s) : \text{Tr}(g) \not\leq \text{Tr}(p)\} \cup \{g\}.$$

$F(s)$ need not be a full-dimensional Pareto set. Its required property is *coverage*: every $a \in C(s)$ has some $p \in F(s)$ with

$\text{Tr}(p) \leq \text{Tr}(a)$. The goal frontier is $T = F(s_{\text{goal}})$. Because OPEN is ordered by f and $h(s_{\text{goal}}) = 0$, the first coordinate of T never decreases, so a path is goal-dominated iff T truncation-dominates its f -value.

3 FAST-MVH

Algorithm 1 gives the node lifecycle. Extract a node. If the goal frontier refutes its heuristic, select the next one and reinsert the node. Otherwise test it locally, expand it, and generate its successors. A successor is first tested for local dominance (line 15) and only then assigned a heuristic (line 16).

Algorithm 1: FAST-MVH

Input: $G, s_{\text{start}}, s_{\text{goal}}$ and a lexicographically ordered admissible MVH H .

Output: a cost-unique Pareto-optimal path set Sols.

```

1 Sols  $\leftarrow \emptyset$ ;  $F(s), R(s) \leftarrow \emptyset$  for all  $s$ 
2  $n \leftarrow (s_{\text{start}}, 0, 1)$ ;  $\text{parent}(n) \leftarrow \perp$ 
3 OPEN  $\leftarrow \{n\}$ 
4 while OPEN  $\neq \emptyset$  do
5    $n \leftarrow \text{OPEN.PopMin}()$ 
6   if GoalDom( $f(n)$ ) then
7      $i \leftarrow \text{ChooseH}(s(n), g(n), i(n) + 1, \text{true})$ 
8     if  $i \neq \perp$  then  $i(n) \leftarrow i$ ; insert  $n$  in OPEN
9     continue
10  if LocalDom( $s(n), g(n)$ ) then continue
11  Update( $s(n), g(n)$ )
12  if  $s(n) = s_{\text{goal}}$  then add  $n$  to Sols; continue
13  for each  $s' \in \text{Succ}(s(n))$  do
14     $g' \leftarrow g(n) + c(s(n), s')$ 
15    if LocalDom( $s', g'$ ) then continue
16     $i' \leftarrow \text{ChooseH}(s', g', 1, \text{false})$ 
17    if  $i' \neq \perp$  then insert  $(s', g', i')$  with parent  $n$  in OPEN
18 return Sols

```

Test order. The two tests at generation are pure: neither changes a frontier. A successor enters OPEN iff it passes both, so their order cannot change which successors enter OPEN or which heuristic they receive. It changes only cost. ChooseH may issue one goal query per heuristic vector, and $|H(s)|$ reaches hundreds of vectors on some of our instances. LocalDom usually decides with one traversal of a small frontier, and on dense instances it rejects most successors. Testing the cheaper and more selective predicate first avoids selecting heuristics for paths that would be discarded anyway.

3.1 Dominance queries

Every frontier is stored in a k-d tree (Bentley 1975). A node v holds a few live vectors $P(v)$, at most two children, and the component-wise minimum $\ell(v)$ and maximum $u(v)$ of the vectors below it. An unindexed array is a single node. Find($\mathcal{K}, q, \kappa$) enumerates the members p of a tree or forest $\mathcal{K}$ with $\kappa(p) \leq q$, where κ is Tr or the identity. It skips a subtree when $\kappa(\ell(v)) \not\leq q$, because then no vector below v qualifies. Callers stop it at the first useful answer.

Algorithm 2: Dominance queries and update

```

1 function Find( $\mathcal{K}, q, \kappa$ )
2    $\Sigma \leftarrow \text{roots of } \mathcal{K}$ 
3   while  $\Sigma \neq \emptyset$  do
4     pop  $v$  from  $\Sigma$ ; if  $\kappa(\ell(v)) \not\leq q$  then continue
5     yield each  $p \in P(v)$  with  $\kappa(p) \leq q$ 
6     push the children of  $v$  on  $\Sigma$ 
7 function GoalDom( $f$ )
8   return true iff Find( $T, \text{Tr}(f), \text{Tr}$ ) yields
9 function LocalDom( $s, g$ )
10   $w \leftarrow \text{NONE}$ 
11  for each  $p \in \text{Find}(F(s), \text{Tr}(g), \text{Tr})$  do
12    if  $p_1 \leq g_1$  then return true  $\triangleright$  full witness
13     $w \leftarrow \text{TR}$ 
14  if  $w = \text{NONE}$  then return false
15  return true iff Find( $R(s), g, \text{id}$ ) yields
16 procedure Update( $s, g$ )
17  for each  $p \in F(s)$  with  $\text{Tr}(g) \leq \text{Tr}(p)$  do
18    remove  $p$  from  $F(s)$ 
19    if  $p_1 < g_1$  then append  $p$  to  $R(s)$ 
20  insert  $g$  into  $F(s)$ 

```

Witnesses. A truncated query over $F(s)$ returns one of three answers. If no member truncation-dominates g , coverage implies that no expanded path at s dominates g at all, and the test ends (line 14). If some truncated dominator also satisfies $p_1 \leq g_1$, it is a full-dimensional witness and the test ends at once (line 12). Only when every truncated dominator has a larger first coordinate ($w = \text{TR}$) is an exact full-dimensional search needed (line 15). Capturing full witnesses during the truncated traversal means that $F(s)$ is never scanned twice. It also makes a separate record of $\max_{p \in F(s)} p_1$ unnecessary: if that maximum is at most g_1 , the first truncated dominator found is already a full witness. Update traverses the same tree symmetrically, skipping v when $\text{Tr}(g) \not\leq \text{Tr}(u(v))$.

3.2 The residue

Why is a fallback needed at all? $F(s)$ is an antichain in $M - 1$ coordinates, not in M . Expanding g evicts every p with $\text{Tr}(g) \leq \text{Tr}(p)$, even when $p_1 < g_1$. Such a p is still a valid expanded cost and may dominate a later arrival that g does not. Local dominance therefore cannot be decided from $F(s)$ alone.

FAST-MVH keeps a *residue* $R(s) \subseteq C(s)$. When an expansion with cost g evicts p , p enters $R(s)$ iff $p_1 < g_1$ (line 19). Otherwise $g \leq p$ in all M coordinates, g answers every query p could answer, and p is discarded. Vectors never leave $R(s)$, and $C(s)$ itself is never stored.

Example. Let $a = (3, 3, 3)$ be expanded at s , and later $b = (9, 2, 2)$. Update evicts a , since $\text{Tr}(b) \leq \text{Tr}(a)$, and as $3 < 9$, a enters $R(s)$. A new path $g = (4, 3, 3)$ has the truncated dominator b but no full dominator in $F(s)$; line 15 finds a . Had a been $(9, 3, 3)$ and $b = (3, 2, 2)$, then $b \leq a$, any g dominated by a is dominated by b , and line 12 finds b .

3.3 Heuristic selection

ChooseH(s, g, j, b) returns the first index $i \geq j$ whose vector the goal frontier does not refute, or $\perp$. Its outer loop is that of Wolff et al.: candidates are scanned in the lexicographic order fixed by $H(s)$, and the first unrefuted one is taken, because that order is what makes $i(n)$ monotone non-decreasing along any OPEN entry and admissibility carry through dimensionality reduction. FAST-MVH changes nothing about which index is returned. What it changes is how many of the $k(s)$ candidates a call actually tests. The *redundancy pointer* $\rho_s(i)$ is the largest $i' < i$ with $\text{Tr}(h^{i'}) \leq \text{Tr}(h^i)$, or 0 if none exists, and is computed once per state, when $H(s)$ is first used, by a single lexicographic pass over $\text{Tr}(H(s))$. It is this pointer, together with the reordering described after Algorithm 1, that the baseline does not have: Wolff et al.’s ChooseH issues one GoalDom query per candidate until it finds an unrefuted one, in both the goal-promotion and the generation call; FAST-MVH issues a query only for a candidate not already known to be refuted by an earlier, truncation-dominating candidate.

Algorithm 3: ChooseH(s, g, j, b)

Precondition: b is true only if h^{j-1} was just refuted by GoalDom.

```

1 if  $j > k(s)$  then return  $\perp$ 
2 if  $T = \emptyset$  then return  $j$ 
3  $r \leftarrow j - 1$  if  $b$  else  $j$ 
4 for  $i = j, \dots, k(s)$  do
5   if  $\rho_s(i) \geq r$  then continue  $\triangleright$  refuted via  $h^{\rho_s(i)}$ 
6   if not GoalDom( $g + h^i$ ) then return  $i$ 
7 return  $\perp$ 
```

Candidates are examined in lexicographic order, as in Wolff et al. Line 5 skips only candidates already known to be refuted; omitting a pointer loses a shortcut, never an answer, so Algorithm 3 is a strict refinement of the baseline’s ChooseH: same inputs, same output, fewer queries. On states where $H(s)$ holds hundreds of truncation-redundant candidates, a single call can fall from $O(k(s))$ queries to $O(1)$, each of which would otherwise have walked the goal frontier; Section 5.3 returns to why indexing $H(s)$ itself does not improve on this further.

4 Correctness

Lemma 1 (Residue sufficiency). Suppose no $p \in F(s)$ satisfies $p \leq g$. Then some $a \in C(s)$ satisfies $a \leq g$ iff some $r \in R(s)$ does.

Proof. $R(s) \subseteq C(s)$ gives one direction. For the other, let $a_0 \in C(s)$ with $a_0 \leq g$. By assumption $a_0 \notin F(s)$, so some later expansion b evicted it, with $\text{Tr}(b) \leq \text{Tr}(a_0)$. If $a_0 \in R(s)$ we are done. Otherwise $b_1 \leq (a_0)_1$, so $b \leq a_0 \leq g$; let $a_1 = b$ and repeat. Each step moves to a strictly later expansion, so the chain is finite. It cannot end in $F(s)$, by assumption, so it ends in $R(s)$. $\square$

Lemma 2 (Local exactness). LocalDom(s, g) returns true iff some $a \in C(s)$ satisfies $a \leq g$.

Proof. Coverage holds initially and is preserved by Update, since the new member truncation-dominates every vector it evicts and $\leq$ is transitive. If Find yields nothing, no $a \in C(s)$ dominates even $\text{Tr}(g)$. A return at line 12 exhibits $p \in F(s) \subseteq C(s)$ with $p \leq g$. Otherwise every truncated dominator in $F(s)$ was examined and none is full, so Lemma 1 applies at line 15. Box pruning discards only subtrees with no qualifying vector. $\square$

Lemma 3 (Safe skipping). Line 5 of Algorithm 3 skips only refuted candidates.

Proof. T changes only through Update, which preserves coverage, so a refuted truncated query stays refuted. Every index in $[r, i)$ has been refuted, during this call or, when b holds, by the goal test that preceded it. If $\rho_s(i) \geq r$, then $\text{Tr}(g + h^{\rho_s(i)}) \leq \text{Tr}(g + h^i)$, so the vector that refuted $h^{\rho_s(i)}$ also refutes h^i . $\square$

Theorem 1. With identical heuristic arrays, successor order, and OPEN tie-breaking, FAST-MVH makes the same OPEN insertions and extractions as $L\text{-NAMOA}_{dr}^*\text{-mvh}$ and returns the same solutions in the same order.

Proof. Induct on extractions. GoalDom is exact on the same T . Lemma 2 makes LocalDom exact. By Lemma 3, ChooseH returns the first unrefuted index. The two tests at generation are pure, so their order preserves both their conjunction and the selected index. Hence the same nodes enter OPEN in the same order. $\square$

Completeness and Pareto optimality therefore follow from those of $L\text{-NAMOA}_{dr}^*\text{-mvh}$ under its assumptions: an admissible MVH sorted lexicographically.

5 Engineering the Dominance Tests

Theorem 1 fixes what FAST-MVH computes: node-for-node, it is $L\text{-NAMOA}_{dr}^*\text{-mvh}$. Every gain in Section 6 is therefore a constant-factor gain on the same search tree, won by changing four things the baseline does unindexed: (i) frontier queries, which the baseline answers by a linear scan of $F(s)$, become Find traversals of an adaptively promoted k-d tree (5.1); (ii) the full-dimensional fallback, which the baseline answers by rescanning $C(s)$ under a cached maximum, is answered from a provably sufficient residue $R(s)$ indexed by a forest of static trees (5.2); (iii) heuristic selection, which the baseline performs by querying every candidate up to the first unrefuted one, is pruned by the redundancy pointers of Section 3.3 before a query is ever issued; and (iv) node and heuristic storage move from pointer-linked records to contiguous arrays (5.4). None of the four changes the sequence of expansions, insertions, or solutions; each only lowers the constant in front of a cost that the baseline already pays. The following subsections detail each mechanism, including one, heuristic-set indexing, that we implemented and discarded (5.3), and one, full-dimensional fallback caching, that an earlier design of FAST-MVH used and the residue forest has since made obsolete (5.2).

5.1 Adaptive frontiers

Most states hold a handful of vectors, and a scan of a short contiguous array beats any tree: it has no pointer indirection, and the hardware prefetcher hides its memory latency. Trees pay only on frontiers that are both large and frequently queried. Each $F(s)$ therefore starts as a contiguous array and records the mean number of comparisons its queries cost. After 16 queries, a frontier with at least 8 live vectors and a mean scan of at least 64 comparisons is promoted, once, to a dynamic k-d tree over its last $M - 1$ coordinates. Nodes live in an index-addressed pool and store both corners $\ell(v)$ and $u(v)$, so the same tree serves dominance queries (pruning on ℓ) and updates (pruning on u). Evicted vectors are marked dead rather than removed; the tree is rebuilt when its size exceeds $1.25b + 64$, where b is its live count at the last build.

The rule tracks measured cost rather than M . On a 10×10 grid with three objectives no frontier is ever promoted, while a 16,000-vertex road graph with three objectives promotes eleven. Promotion is the mechanism by which geometric indexing is applied exactly where the antichain lower bound of Shperberg et al. bites and nowhere else.

5.2 The residue forest

The fallback is rare per query but not rare per search. On dense instances $R(s)$ grows to between 60% and 93% of all expanded vectors (Section 6.2), and in the worst cases tens of thousands of fallback queries reach it. A linear scan would cost $|R(s)|$ comparisons each time, and $R(s)$ only grows. The fallback must therefore be indexed.

$R(s)$ is append-only, so it suits the logarithmic method of Bentley and Saxe (1980). New vectors are appended to an unindexed buffer. Nothing is built until a query finds 64 buffered vectors. The buffer then becomes a static implicit k-d tree: a contiguous block permuted so that each median sits at the middle of its range, with axes cycling through all M coordinates and a lower corner stored per subtree. A block is merged with its predecessor while the predecessor is less than twice its size, so a state holds $O(\log|R(s)|)$ blocks and each vector is rebuilt $O(\log|R(s)|)$ times. A query scans the buffer and then searches the blocks, which are fixed and contiguous. States that never fall back never build a tree, so instances without fallbacks pay nothing for the mechanism. The effect is large where the residue is large: on Grid 6×6 , $\rho = -0.2$ at eight objectives, 2,203,027 full-dimensional comparisons answer 10,701 fallback queries against a residue that has absorbed 74,271 vectors by the end of the run — 206 comparisons per query against a forest that a linear scan would need to walk almost entirely (Table 2).

Remark (fallback caching). An earlier design of FAST-MVH answered the fallback by scanning all of $C(s)$, guarded by an exactly maintained maximum first coordinate of $F(s)$, and cached for each candidate the vector that had last refuted it, so that a repeated candidate could be rejected without a scan. The cache was effective, about 10–18% faster on the largest instances, because fallback queries recur at the same states with similar costs. It reduced the number of scans, however,

not their length: every miss still paid $|C(s)|$ comparisons. The residue forest removes the cause rather than the symptom. Every fallback query is now exact and sub-linear, the residue excludes both the live frontier and vectors dominated by their evictor, and neither the cache nor the separate maximum earns its keep. Neither is part of the method evaluated here.

5.3 Heuristic sets are not indexed

It is natural to index $H(s)$ as well, and we implemented such an index: a k-d tree over $\text{Tr}(H(s))$ with subtree corners ℓ, u . Its rules are sound. All members of a subtree are refuted if $\text{GoalDom}(g + \ell)$; the first member is accepted if $\neg \text{GoalDom}(g + u)$. In every configuration we tested, including instances whose sets average several hundred vectors per state, it did not reduce search time, and it is disabled in the evaluated method. Three facts explain this.

First, a box test over $H(s)$ is not a cheap comparison. Each corner test is itself a goal-frontier query. Second, the corners are not attainable heuristic values. On an anti-correlated Pareto surface the lower corner is dominated far more easily than any member, and the upper corner far less easily (Figure 1), so both rules rarely decide. A frontier box, by contrast, is tested against a single query point, and its corner is tight in the coordinates that matter. Third, the work the index could save has largely been removed already. Local-first ordering eliminates most calls to ChooseH; redundancy pointers skip many refuted candidates without a query; and ChooseH must return the *first* unrefuted index, so an index may only discard whole prefixes and can never jump ahead. The remaining selections scan a short contiguous array of truncated vectors, which is the access pattern the hardware handles best.

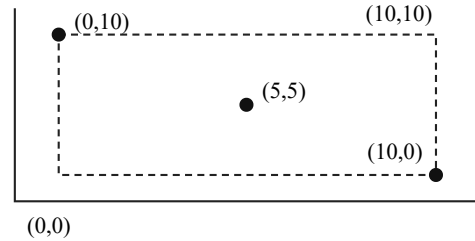


Figure 1. A heuristic box contains unattainable corners. For $g = 0$, goals at the three members refute every member but not the lower corner, so the box cannot be rejected. A goal at $(6,6)$ refutes the upper corner but no member, so no member can be accepted early.

5.4 Node storage

OPEN entries hold f inline together with the index of a record that stores g , the state, the parent, and the heuristic index. Records are appended to one contiguous array. The heap comparator, the lexicographic order on f , therefore never follows a pointer, and generating a node allocates nothing. Solution paths are materialized once, at the end. The truncated vectors of each $H(s)$ are likewise stored contiguously. These choices change no decision; they reduce the constant factor of every operation in Algorithm 1, and they matter most at three objectives, where queries are short and node handling dominates.

6 Experiments

Setup. All runs use one machine (Intel Core i5-1135G7, 16 GB, Windows 11) and GCC 16.2 with `-O2 -DNDEBUG -std=c++20`. The baseline is the implementation of $L\text{-NAMOA}_{dr}^*\text{-mvh}$ by Wolff et al. (unmodified source, commit 0a2f9ea). Times are search times, excluding input parsing. FAST-MVH times are minima of three runs interleaved with the baseline; the baseline runs once, or three times when it takes under 5 s. We report only instances that the baseline solves within six minutes on this machine. In all seventeen, both solvers returned byte-identical solution files and the same numbers of solutions, expansions, and extractions, as Theorem 1 predicts.

Instances. Grids are four-connected $n \times n$ graphs with integer costs; ρ is the pairwise correlation of the objectives, and the 10×10 grids use cyclic trade-off costs. Road instances are breadth-first subgraphs of the DIMACS New York and Bay Area networks (Demetrescu, Goldberg, and Johnson 2009), grown from the network’s central vertex, which serves as the goal. *NY- k* and *Bay- k* denote subgraphs with k thousand vertices: Bay-8 and Bay-16 have 8,000 and 16,000 vertices, and Bay-0.8 has 800. The first two road objectives are distance and travel time; the others are uniform integers in $[1, 100]$. Heuristics are A*pex backward Pareto sets (Zhang et al. 2022) with approximation factor ε , sorted lexicographically within each state as Section 2 requires.

Table 1. Search time on seventeen instances, ordered by the number of objectives M . $|\Pi|$ is the number of Pareto-optimal solutions. Times in seconds. Speedup is baseline time divided by FAST-MVH time. Solutions, expansions, and solution files are identical for both solvers.

Instance	M	ε	$ \Pi $	Expansions	$L\text{-NAMOA}_{dr}^*\text{-mvh}$	FAST-MVH	Speedup
Grid 10×10	3	0	1,602	9,500	0.058	0.032	1.8
Bay-8	3	0.1	238	12,346	0.053	0.018	2.9
Bay-16	3	0.1	3,534	347,498	2.67	0.647	4.1
NY-3	4	0.05	646	10,962	0.132	0.029	4.6
Grid 10×10	4	0.05	11,330	53,652	1.74	0.338	5.1
NY-5	4	0.01	7,105	144,117	9.55	1.04	9.2
Bay-8	4	0.1	3,716	157,704	20.1	0.654	30.7
NY-8	4	0.05	5,713	195,594	39.1	1.03	37.9
Grid 8×8 , $\rho = 0$	5	0.05	9,412	33,005	1.47	0.241	6.1
Bay-0.8	5	0.1	7,617	34,871	5.98	0.216	27.7
NY-5	5	0.05	26,701	503,505	184	5.46	33.7
NY-3	6	0.05	11,286	127,349	104	1.27	82.0
Grid 8×8 , $\rho = 0$	6	0.05	70,196	179,297	92.0	2.34	39.4
Grid 8×8 , $\rho = -0.2$	6	0.05	104,480	293,896	321	5.85	54.8
Grid 7×7 , $\rho = -0.2$	7	0.05	32,643	91,704	24.5	1.38	17.7
Grid 7×7 , $\rho = -0.4$	7	0.05	49,884	132,959	57.6	2.51	23.0
Grid 6×6 , $\rho = -0.2$	8	0.05	47,079	99,244	37.0	1.44	25.8

6.1 Search time

FAST-MVH is faster on every instance. With three objectives the gain is 1.8 to 4.1. Truncation leaves two coordinates, frontier queries are short scans, and the 10×10 grid promotes no frontier at all; the gain here comes mainly from node storage and from local-first ordering. Bay-16 already promotes eleven frontiers, and its gain is the largest at this dimension.

From four objectives on, the gain grows to between 4.6 and 82. Road networks gain most. With four objectives, Bay-8 and NY-8 gain 31 and 38 times, and with six, NY-3 gains 82 times. On these instances few vectors per state are dominated in all coordinates, frontiers become long antichains, and every unindexed query scans them in full. The k-d trees replace these scans by box-pruned traversals, and local-first ordering prevents most heuristic selections.

The gain is not a function of M alone. Grid 8×8 in five objectives returns more solutions than NY-5 in four, yet gains 6.1 against 9.2. With seven and eight objectives the grids gain 18 to 26 times, less than the six-objective grids (39 and 55). In these small, dense grids many paths reach each state, residues are large, and a larger fraction of the work lies in the fallback (Section 6.2). Across the table, the gain follows the frontier workload of an instance, its frontier lengths and query counts, rather than its

dimension or its number of solutions. The largest absolute saving is on Grid 8×8 with $\rho = -0.2$ in six objectives: 321 s become 5.9 s.

Table 2. The full-dimensional fallback, on the same seventeen instances as Table 1. Fallback queries are local tests that reach line 15 of Algorithm 2; Rate is that count divided by expansions. $|R|/\text{Exp.}$ is the final total size of all residues divided by the number of expansions — how much of the search the residue forest ends up holding. Comparisons/query is the mean number of full-vector comparisons an indexed fallback query performs. Speedup repeats Table 1’s column, so the fallback profile of an instance can be read against its overall gain without turning back a page.

Instance	M	Expansions	Fallback queries	Rate	$ R /\text{Exp.}$	Comparisons/query	Speedup
Grid 10×10	3	9,500	3,806	40.1%	0.93	29.5	1.8
Bay-8	3	12,346	162	1.3%	0.26	9.5	2.9
Bay-16	3	347,498	5,783	1.7%	0.67	24.9	4.1
NY-3	4	10,962	101	0.9%	0.07	3.5	4.6
Grid 10×10	4	53,652	11,778	22.0%	0.79	68.7	5.1
NY-5, $\varepsilon = 0.01$	4	144,117	58,440	40.6%	0.75	47.1	9.2
Bay-8	4	157,704	6,414	4.1%	0.12	8.0	30.7
NY-8	4	195,594	5,439	2.8%	0.24	28.6	37.9
Grid 8×8 , $\rho = 0$	5	33,005	3,164	9.6%	0.58	64.4	6.1
Bay-0.8	5	34,871	0	0%	0	—	27.7
NY-5	5	503,505	91,934	18.3%	0.70	52.6	33.7
NY-3	6	127,349	333	0.3%	0.01	4.4	82.0
Grid 8×8 , $\rho = 0$	6	179,297	14,804	8.3%	0.64	125.7	39.4
Grid 8×8 , $\rho = -0.2$	6	293,896	39,359	13.4%	0.67	124.5	54.8
Grid 7×7 , $\rho = -0.2$	7	91,704	7,418	8.1%	0.49	80.2	17.7
Grid 7×7 , $\rho = -0.4$	7	132,959	13,387	10.1%	0.60	96.3	23.0
Grid 6×6 , $\rho = -0.2$	8	99,244	10,701	10.8%	0.75	205.9	25.8

6.2 The fallback

Table 2 shows that the fallback depends on the instance far more than on M , and that it takes three distinct forms, each with a different relationship to the speedup in Table 1.

No fallback. On Bay-0.8 in five objectives no local test ever reaches line 15: every truncated dominator found is also a full witness, no residue is ever created, and no residue tree is ever built. The 27.7-fold gain on this instance is produced entirely by frontier indexing and test order, at zero cost from the mechanism of Section 5.2 — the lazy, query-triggered construction means a state that never falls back never pays for one.

Rare fallback. On NY-3 in six objectives, the single largest speedup in the paper, only 333 of 127,349 expansions reach the fallback at all, and the residue holds just 1% of expanded vectors; each of those 333 queries costs 4.4 comparisons. Bay-8 at four objectives and NY-3 at four objectives are similar: under 5% of expansions fall back, each for single-digit comparisons. On these instances the fallback is not where the time goes — it is a cheap exactness guarantee riding on top of a gain that comes almost entirely from frontier indexing (5.1) and from local-first ordering keeping ChooseH off the critical path.

Heavy fallback. On the dense grids and on NY-5, 8% to 41% of expansions carry a fallback, and the residue absorbs 58–93% of all expanded vectors by the end of the run. These are exactly the cases Section 5.2 is built for: without the residue forest, each of the tens of thousands of fallback queries on Grid 8×8 , $\rho = -0.2$ at six objectives would scan a residue that has grown to 198,192 vectors; the forest answers each in 124.5 comparisons, a sub-linear cost that holds even as the residue climbs into the hundreds of thousands. The mean cost per query still grows with M — from 29.5–68.7 comparisons at three and four objectives to 96.3–205.9 at seven and eight — because a k-d tree’s pruning power weakens as dimension grows; this is the main reason the seven- and eight-objective grids gain 17.7 to 25.8-fold while the six-objective ones reach 39.4 and 54.8-fold on comparable instances.

Test order and fallback counts. Local-first ordering can increase the number of fallbacks. A successor that heuristic selection would discard now reaches the local test first, and may trigger a fallback on its way to being rejected. Theorem 1 preserves decisions, not the number of calls to each test. On every instance in Table 2 the trade is favourable: a fallback query costs at most

a few hundred comparisons, while a heuristic selection skipped by local-first ordering may otherwise have issued a goal query for each of hundreds of candidate vectors — the asymmetry Section 3.3 exploits with the redundancy pointer.

Reproducibility. All runs are stored in `benchmarks/runs/` under the suffixes `fast_m3_6`, `fast_ny_lex`, `ny_sample`, and `fast_m5_8_home_instances`. Each run directory holds the exact commands, the commit, binary hashes, solution files, and every counter reported here, including `num_full_dominance_check`, `num_fallback_indexed`, and `cmp_full`, from which Table 2 is computed exactly, not estimated. The Bay instances share one goal, and the extra road objectives are synthetic, so the instances are not independent samples of routing problems.

7 Conclusion

Dominance checking, not path generation, limits multi-objective search with multi-valued heuristics as the number of objectives grows. FAST-MVH reduces its cost with three mechanisms: adaptive k-d trees over truncated frontiers, a single-pass local test applied before heuristic selection, and an exact fallback confined to a provably sufficient residue indexed by a forest of static k-d trees. Indexing the heuristic sets themselves does not pay, because each box test is a goal query and the boxes of anti-correlated trade-offs are loose. FAST-MVH expands exactly the paths of $L\text{-NAMOA}_{dr}^*\text{-mvh}$ and returns identical solutions. On seventeen instances with three to eight objectives it ran 1.8 to 82 times faster, and the gain followed the frontier workload of each instance rather than its dimension.

Three directions remain open. Road networks with seven and eight objectives would test the method where both frontiers and residues are largest. MVHs that are not ordered lexicographically require separate admissibility arguments (Skyler et al. 2024). Finally, the per-fallback cost grows with M ; geometric indexes with stronger high-dimensional pruning, or a residue that is periodically reduced to its full-dimensional Pareto set, could lower it further.

References

- Bentley, J. L. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM* 18(9): 509–517.
- Bentley, J. L.; and Saxe, J. B. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1(4): 301–358.
- Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series 74. American Mathematical Society.
- Geisser, F.; Haslum, P.; Thiébaux, S.; and Trevizan, F. 2022. Admissible Heuristics for Multi-Objective Planning. In *Proceedings of ICAPS*, 100–109.
- Shperberg, S. S.; et al. A Geometric Index for Multi-Objective Dominance Checking. Manuscript under review.
- Skyler, S.; Shperberg, S. S.; Atzmon, D.; Felner, A.; Salzman, O.; Chan, S.; Zhang, H.; Koenig, S.; Yeoh, W.; and Hernández Ulloa, C. 2024. Theoretical Study on Multi-Objective Heuristic Search. In *Proceedings of IJCAI*, 7021–7028.
- Wolff, M.; Felner, A.; and Salzman, O. 2026. Bridging Multi-Valued Heuristics and Dimensionality Reduction in Multi-Objective Search. In *Proceedings of SoCS*.
- Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernández Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A*pex: Efficient Approximate Multi-Objective Search on Graphs. In *Proceedings of ICAPS*, 394–403.